OFFICIAL WRITEUP

KNIFE

An easy Linux machine exploiting a backdoored PHP 8.1.0-dev development build for remote code execution, followed by privilege escalation through a misconfigured sudo rule on the Chef InSpec "knife" utility.

DIFFICULTY: EASY

CATEGORY: LINUX / WEB

OS: UBUNTU

PREPARED BY
T1taX

PLATFORM
Hack The Box

TOC Table of Contents

00 Machine Overview

Machine Name	Knife
Platform	Hack The Box
Difficulty	Easy
Target IP	10.129.41.223
Primary Services	SSH (OpenSSH 8.2p1) — Port 22, Apache HTTP (2.4.41, Ubuntu) — Port 80
Author	T1taX

Knife is an easy-rated Linux machine named after the privilege escalation vector that closes it out. The web server is running a PHP development build that, for a few days in March 2021, shipped with a malicious backdoor after an attacker compromised the PHP source repository directly — giving unauthenticated remote code execution to anyone who knew the trigger. From the resulting low-privilege shell, a permissive sudo rule allows the local user to run `/usr/bin/knife` — Chef's command-line management tool — as root with no password, which trivially escalates to a full root shell.

> **Objective:** Exploit the backdoored PHP build for an initial foothold, then abuse a misconfigured sudo rule on `knife` to escalate to root.

TOOLS USED

`nmap` `public PoC (PHP 8.1.0-dev backdoor)` `python3` `sudo -l`

TECHNIQUES

`Public-Facing Application Exploitation` `Supply-Chain Backdoor Abuse` `Sudo Misconfiguration Abuse`

ATTACK CHAIN

RECON → **BACKDOOR ID** → **RCE FOOTHOLD** → **SUDO ABUSE** → **ROOT**

01 Reconnaissance

An Nmap scan with default scripts and version detection finds only two open ports:

```
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubun
| ssh-hostkey:
|   3072 be:54:9c:a3:67:c3:15:c3:64:71:7f:6a:53:4a:4c:21 (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCjEtN3+WZzlvu54zya9Q+D0d/jwjZT2jY
I1Q3JYOH41efTbGDFHiGSTY1lH3HcAvOFh75dCID0564T078p7ZEIoKRt1l7Yz+GeMZ870Nw1
2Xg0hCE6H03Ri6GtYs5xVFw/KfGSGb7OJT1jhitbpUxRbyvP+pFy4/8u6Ty91s98bXrCyaEy2
Nm0CpiNfoHBCBDJ1LR8p8k=
|   256 bf:8a:3f:d4:06:e9:2e:87:4e:c9:7e:ab:22:0e:c0:ee (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAAB
|   256 1a:de:a1:cc:37:ce:53:bb:1b:fb:2b:0b:ad:b3:f6:84 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJbkxEqMn++HZ2uEvM0lDZy+TB8B8IAeWRB
80/tcp open  http    syn-ack ttl 63 Apache httpd 2.4.41 ((Ubuntu))
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-title:  Emergent Medical Idea
|_http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Fig 1.1 — Nmap scan: OpenSSH 8.2p1 and Apache 2.4.41 (Ubuntu)

MITRE ATT&CK: T1046 — Network Service Discovery

DETECTION NOTES

A default two-port scan against a host is low-noise, but Nmap's `-sC` script probes (banner grabs, HTTP title fetch) still generate a handful of distinctive, malformed-looking requests in the web server's access log within the same second — a useful low-fidelity indicator when correlated with a single source IP touching multiple services back-to-back.

Port `80` hosts a site titled "Emergent Medical Idea," served by Apache `2.4.41` on Ubuntu — a standard-looking web application at first glance.

02 Vulnerability Identification

The web server's response headers disclose the PHP version powering the site: `PHP/8.1.0-dev`. This specific version string is a red flag on its own — `8.1.0-dev` is a nightly development build that should never appear on a production server. It matches a well-known supply-chain incident from March 2021, in which attackers compromised the PHP source repository's git server directly and pushed two malicious commits into the master branch, inserting a backdoor before the tampering was caught and reverted within hours.

> **The backdoor:** The injected code inspects incoming requests for an HTTP header literally named `User-Agentt` (the extra "t" is intentional — it avoids colliding with the legitimate `User-Agent` header). If present, its value is passed to PHP's `zend_eval_string()`, executing it as raw PHP code — full unauthenticated remote code execution, hidden in plain sight inside what looks like an ordinary request header.

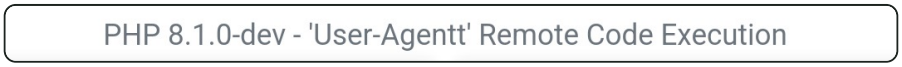

Fig 2.1 — Public PoC reference: "PHP 8.1.0-dev 'User-Agentt' Remote Code Execution"

03 Exploitation

Manually crafting the `User-Agentt` header ran into a practical snag: any payload containing a single or double quote got mangled by PHP's own string handling inside the backdoor's eval call, breaking the injected code and returning a generic `No input file specified` error instead of executing anything useful — and quotes are hard to avoid once a payload needs to call a function with arguments. Rather than fight that manually, a public, ready-made PoC script was used instead (`flast101/php-8.1.0-dev-backdoor-rce`), which builds the header correctly and wraps the whole thing in an interactive pseudo-shell:

```
python3 49933.py
```

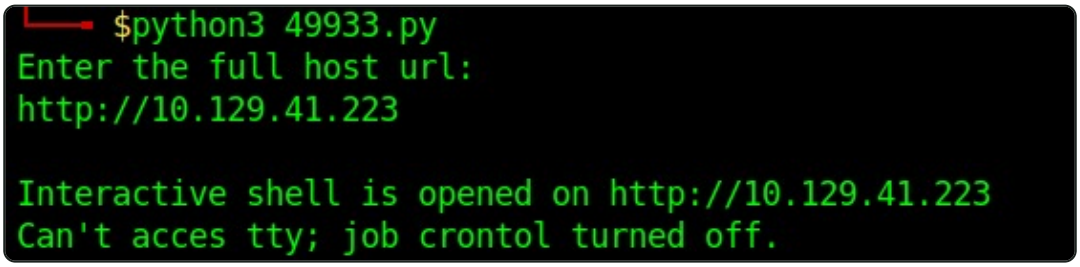

Fig 3.1 — Running the public PoC against the target, opening an interactive shell over HTTP

MITRE ATT&CK: T1190 — Exploit Public-Facing Application

DETECTION NOTES

A WAF or reverse-proxy rule that flags requests containing a header name it doesn't recognize — particularly a near-duplicate of a standard header like `User-Agentt` — would catch this immediately. In the absence of a WAF, this is still visible in raw Apache access logs as a request with an unusual, non-standard header field, and any resulting outbound connection or command execution from the `www-data` process is a strong EDR/auditd signal (a web server process spawning a shell is almost never legitimate).

Result: The pseudo-shell executes as the low-privileged web service account, providing an initial foothold on the host.

04 Foothold & User Flag

The shell obtained through the backdoor is stable enough to browse the filesystem. The user flag is located directly in a local user's home directory:

```
cat /home/james/user.txt
```

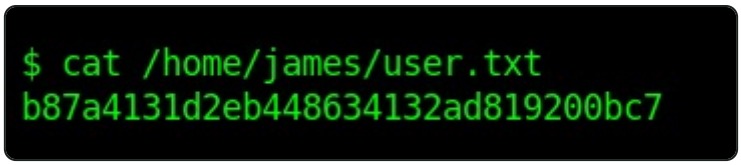

Fig 4.1 — Reading user.txt from james' home directory

```
b87a4131d2eb448634132ad819200bc7
```

05 Privilege Escalation

With a session as `james`, the account's sudo rights are checked:

```
sudo -l
```

Fig 5.1 — james may run /usr/bin/knife as root with no password required

`knife` is Chef Infra's command-line tool for managing nodes, cookbooks, and — critically — for running arbitrary Ruby code against the local Chef configuration through its exec subcommand. Since it's allowed to run as root via `NOPASSWD`, that Ruby execution capability can be pointed straight at a shell:

```
sudo /usr/bin/knife exec -E 'exec "/bin/bash"'
```

Fig 5.2 — Using knife's Ruby exec capability to spawn a root-owned bash shell

MITRE ATT&CK: T1548.003 — Abuse Elevation Control Mechanism: Sudo and Sudo Caching

DETECTION NOTES

Every invocation of a NOPASSWD sudo rule is still recorded by `auditd` / the system's sudo log (typically forwarded to `/var/log/auth.log` or a SIEM). A rule alerting on sudo executions of `knife exec` — or more generally, any allow-listed sudo binary spawning an interactive shell as its child process — would catch this technique regardless of which GTFOBins-style binary is being abused.

Result: The spawned shell runs as `root`, completing the privilege escalation.

06 Root Flag Captured

```
cat /root/root.txt
```

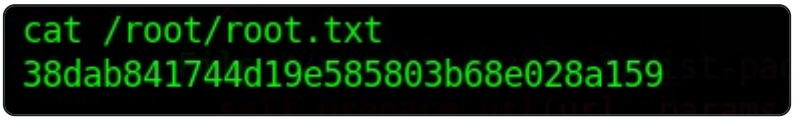

Fig 6.1 — Reading root.txt as root

```
38dab841744d19e585803b68e028a159
```

07 Key Takeaways

Knife ties together a supply-chain compromise with a classic sudo misconfiguration. Neither piece is exotic on its own — running a development build in production, and granting NOPASSWD sudo on a tool capable of running arbitrary code — but together they collapse into an unauthenticated-to-root chain in two steps.

Finding	Risk	Remediation
Production web server running a PHP development build (8.1.0-dev) containing a known backdoor	**Critical**	Never deploy -dev/nightly builds to production; pin to official, signed release versions and monitor upstream security advisories for the exact version in use.
`james` can run `/usr/bin/knife` as root with NOPASSWD, and knife can execute arbitrary Ruby	**High**	Avoid granting sudo access to management tools capable of arbitrary code execution (check GTFOBins before writing any sudoers rule); if knife must run as root, restrict it to specific, non-interactive subcommands.